

DAY 5

Data Structures in Python

Lists · Tuples · Sets · Dictionaries

Searching · Sorting · Comprehensions · Mini Project

[LIST]

(TUPLE)

{ SET }

{ DICT: }



Today's Agenda

01

Intro to Data Structures

Mutable vs Immutable, Memory basics

02

Lists

CRUD, Methods, Comprehension, Deep Copy

03

Tuples

Immutability, Packing/Unpacking, Use-cases

04

Sets

Unique values, Set operations, Frozen sets

05

Dictionaries

Key-value, Methods, Comprehension, Merging

06

Comparison & Searching

When to use which, sorted(), key param

07

Practical Applications

Student Records, Contact Book, Shopping Cart

08

Mini Project

Smart Inventory Management System



What are Data Structures?

A Data Structure is a way of organizing, storing, and managing data in memory so it can be accessed and modified efficiently. Python has four built-in collection data structures — each designed for a different purpose.

[]

LIST

Mutability

Mutable

Order

Ordered

Duplicates

Duplicates ✓

Best for

Store ordered sequence of items

()

TUPLE

Mutability

Immutable

Order

Ordered

Duplicates

Duplicates ✓

Best for

Fixed, read-only data records

{ }

SET

Mutability

Mutable

Order

Unordered

Duplicates

Unique only

Best for

Unique items, fast lookup & set math

{ : }

DICT

Mutability

Mutable

Order

Ordered*

Duplicates

Keys unique

Best for

Key-value pairs, fast access by key

* Dictionaries maintain insertion order since Python 3.7+



Mutable vs Immutable Objects

Understanding mutability is fundamental to using Python data structures correctly. It determines whether an object's value can change after creation.

✓ MUTABLE — Can be changed after creation

Examples: list, dict, set

```
fruits = ['apple', 'banana', 'mango']
fruits[0] = 'cherry' # ✓ works!
fruits.append('grape') # ✓ OK
print(fruits)
# ['cherry', 'banana', 'mango', 'grape']
```

Key Traits:

- Can update individual elements
- Can add or remove elements
- `id()` stays same after modification

🔒 IMMUTABLE — Cannot be changed

Examples: tuple, str, int, float, bool

```
coords = (10, 20, 30)
# coords[0] = 99 ← TypeError!
# Cannot modify a tuple

name = 'Python'
# name[0] = 'J' ← TypeError!
name = 'JPython' # creates NEW str
```

Key Traits:

- Cannot modify elements in-place
- Safer to use as dict keys or set members
- Generally faster and uses less memory



Lists

Ordered · Mutable · Allow Duplicates · Indexed from 0



Lists — Creating, Indexing & Slicing

```
# — Creating Lists —  
empty = []           # empty list  
nums = [10, 20, 30, 40, 50] # integers  
mixed = [1, "hello", 3.14, True] # mixed types  
nested = [[1,2], [3,4], [5,6]] # nested list  
from_range = list(range(1, 6)) # [1, 2, 3, 4, 5]
```

List Memory Layout → `nums = [10, 20, 30, 40, 50]`



```
# — Indexing & Slicing —  
print(nums[0])    # 10    (first element)  
print(nums[-1])   # 50    (last element)  
print(nums[1:4])  # [20, 30, 40] (index 1 to 3)  
print(nums[:3])   # [10, 20, 30] (first 3)  
print(nums[2:])   # [30, 40, 50] (from index 2)  
print(nums[::2])  # [10, 30, 50] (every 2nd element)  
print(nums[::-1]) # [50, 40, 30, 20, 10] (reversed)  
# Syntax: list[start : stop : step]
```



Lists — Adding & Removing Elements

`.append(x)`

Adds x to the END of the list. Modifies in-place.

```
fruits.append('mango')    # ['apple', 'banana', 'mango']
```

`.extend([x])`

Appends ALL items from an iterable to the end.

```
fruits.extend(['grape', 'kiwi']) # adds 2 items
```

`.insert(i,x)`

Inserts x at position i. Shifts existing elements right.

```
fruits.insert(1, 'cherry') # inserts at index 1
```

`.remove(x)`

Removes FIRST occurrence of x. Raises ValueError if not found.

```
fruits.remove('banana')   # removes 'banana'
```

`.pop(i)`

Removes & RETURNS item at index i. Default removes last.

```
last = fruits.pop()       # removes & returns last
```

`.clear()`

Removes ALL items from the list. List becomes [].

```
fruits.clear()            # []
```



Lists — Utility Methods & Operations

```
nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]

# count() — frequency of a value
print(nums.count(1))      # 2   (1 appears twice)
print(nums.count(5))      # 2

# index() — first occurrence position
print(nums.index(9))      # 5   (9 is at index 5)
print(nums.index(4))      # 2

# sort() — sorts in-place (modifies list directly)
nums.sort()               # ascending: [1,1,2,3,3,4,5,5,6,9]
nums.sort(reverse=True)   # descending: [9,6,5,5,4,3,3,2,1,1]

# reverse() — reverses list in-place
nums.reverse()            # flips current order

# copy() — shallow copy (independent list, same inner values)
a = [1, 2, 3]
b = a.copy()              # b is a NEW list; modifying b won't affect a
b.append(99)
print(a)                  # [1, 2, 3]    <-- unchanged
print(b)                  # [1, 2, 3, 99]

# len(), min(), max(), sum() — built-ins work on lists
print(len(nums), min(nums), max(nums), sum(nums))
```

copy() vs deepcopy()

copy() is shallow — nested lists still share references.
deepcopy() from the copy module creates fully independent copies.

del vs pop() vs remove()

del list[i] removes by index, no return.
pop(i) removes by index, returns value.
remove(x) removes by value.

Sorting tip

sort() changes the list IN-PLACE.
sorted(list) returns a NEW sorted list without changing the original.

List Comprehension Preview

[expr for item in iterable if condition] — creates new list in one line. See next slide!



List Comprehension — The Pythonic Way

List comprehension provides a concise, readable way to create lists. It replaces multiple lines of a for-loop with a single expressive line.

```
[ expression for item in iterable if condition ]
```

```
# — BASIC: squares of 1-10 —————
```

```
squares = [x**2 for x in range(1,11)]  
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

```
# — WITH CONDITION: even numbers only —————
```

```
evens = [x for x in range(1,21) if x % 2 == 0]  
# [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
# — TRANSFORMATION: uppercase names —————
```

```
names = ['alice','bob','charlie']  
upper = [name.upper() for name in names]  
# ['ALICE', 'BOB', 'CHARLIE']
```

```
# — NESTED COMPREHENSION: flatten 2D list —————
```

```
matrix = [[1,2,3],[4,5,6],[7,8,9]]  
flat = [num for row in matrix for num in row]  
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# — EQUIVALENT for-loop (what comprehension replaces) —————
```

```
# squares = []  
# for x in range(1,11):
```



Tuples

Ordered · Immutable · Allow Duplicates · Faster than Lists



Tuples — Basics, Packing & Unpacking

— Creating Tuples

```
empty    = ()           # empty tuple
single   = (42,)        # ← COMMA required for 1-element tuple!
point    = (10, 20)     # 2D coordinate
rgb      = (255, 128, 0) # RGB color
record   = ("Alice", 22, "Jaipur") # mixed types
packed   = 1, 2, 3      # parentheses optional (tuple packing)
```

— Tuple Packing & Unpacking

```
person = ("Bob", 25, "Engineer") # packing: 3 values → 1 tuple

name, age, job = person           # unpacking: 1 tuple → 3 variables
print(name)    # Bob
print(age)     # 25
```

— Extended unpacking with *

```
first, *rest = (1, 2, 3, 4, 5)
print(first)  # 1
print(rest)   # [2, 3, 4, 5]
```

— Swap variables using tuple (Pythonic trick)

```
a, b = 10, 20
a, b = b, a      # swap! No temp variable needed
print(a, b)      # 20 10
```

count() & index()

t.count(val) → how many times val appears
t.index(val) → position of first occurrence

Why immutability matters

Tuples can be used as dict keys and set members because they're hashable. Lists cannot — they're mutable.

Tuple vs List — Quick Rule

Use tuple when data should NOT change:
coordinates, RGB values, DB records, return values from functions.

Performance

Tuples are faster to create, iterate, and use less memory than equivalent lists. Prefer tuples for fixed data.



Sets

Unordered · Mutable · NO Duplicates · Hash-based

Sets — Basics, Methods & Operations

```
# — Creating Sets —————
empty = set()           # NOT {} (that's a dict!)
nums = {1, 2, 3, 4, 5}
mixed = {1, "hello", 3.14}
no_dup = {1, 2, 2, 3, 3, 3}  # → {1, 2, 3}  duplicates removed!

# — Adding Elements —————
s = {10, 20, 30}
s.add(40)                # add single item → {10,20,30,40}
s.update([50,60,70])     # add multiple   → {10,20,30,40,50,60,70}

# — Removing Elements —————
s.remove(10)             # removes 10; raises KeyError if missing
s.discard(99)            # removes 99 if present; NO error if missing (safer)
s.pop()                  # removes & returns a RANDOM element (sets are unordered)

# — Membership Testing (very fast – O(1)) —————
print(3 in nums)         # True
print(9 not in nums)     # True
```

frozenset

Immutable version of set. Can be used as dict key.
fs = frozenset({1,2,3}) # unchangeable

Duplicate Removal Trick

```
names = ['Ana', 'Bob', 'Ana', 'Eve', 'Bob']
unique = list(set(names)) # removes duplicates
```

remove() vs discard()

remove(x) → KeyError if x not present
discard(x) → silently does nothing

Always use discard() when unsure if element exists — it's the safe choice.

Set Operations: A = {1,2,3,4} B = {3,4,5,6}

Union A|B
→ {1,2,3,4,5,6}

Intersect A&B
→ {3,4}

Difference A-B
→ {1,2}

Sym.Diff A^B
→ {1,2,5,6}



Dictionaries

Ordered · Mutable · Key-Value Pairs · Keys must be Unique



Dictionaries — Creating & Accessing

— Creating Dictionaries

```
empty = {} # empty dict
student = {"name": "Alice", "age": 20, "gpa": 3.8}
prices = dict(apple=30, banana=10, mango=50) # using dict()
squares = {x: x**2 for x in range(1,6)} # dict comprehension
```

— Accessing Values

```
print(student["name"]) # "Alice" - direct access
print(student.get("age")) # 20 - safe access
print(student.get("cgpa", 0)) # 0 - default if key missing
# student["phone"] → KeyError! Use .get() for safety
```

— Adding & Updating

```
student["email"] = "alice@python.com" # add new key-value
student["age"] = 21 # update existing key
student.update({"age": 22, "city": "Jaipur"}) # update multiple at once
```

— Removing Items

```
val = student.pop("email") # removes "email" key, returns value
item = student.popitem() # removes & returns LAST inserted (key,val)
del student["city"] # removes "city" key (no return value)
student.clear() # removes ALL key-value pairs → {}
```

Keys must be hashable

Valid keys: str, int, float, tuple
Invalid keys: list, dict, set (unhashable)
Values can be anything.

dict.get() vs d[key]

d[key] raises `KeyError` if key missing.
d.get(key, default) returns default safely.
Always use `.get()` for unknown keys.

Merging Dicts (Python 3.9+)

```
d1 = {"a": 1}; d2 = {"b": 2}
merged = d1 | d2 # {'a': 1, 'b': 2}
d1 |= d2 # in-place merge
```

Dict Comprehension

{key: val for k,v in pairs}
Ex: {name: score for name, score in zip(names, scores)}
Creates a dict from any iterable.



Nested Dictionaries & Dictionary Comprehension

```
# — Nested Dictionary —  
students = {  
    "S001": {"name": "Alice", "marks": {"Math":95, "Science":88, "English":76}},  
    "S002": {"name": "Bob", "marks": {"Math":72, "Science":91, "English":83}},  
    "S003": {"name": "Charlie", "marks": {"Math":85, "Science":79, "English":90}},  
}
```

```
# Accessing nested values  
print(students["S001"]["name"]) # Alice  
print(students["S002"]["marks"]["Science"]) # 91
```

```
# Iterating over nested dict  
for sid, info in students.items():  
    avg = sum(info["marks"].values()) / len(info["marks"])  
    print(f"{info['name']} ({sid}) Avg: {avg:.1f}")
```

```
# Output:  
# Alice (S001) Avg: 86.3  
# Bob (S002) Avg: 82.0  
# Charlie (S003) Avg: 84.7
```

```
# — Dictionary Comprehension —  
# Create dict of name → average marks  
averages = {  
    info["name"]: sum(info["marks"].values())/len(info["marks"])  
    for sid, info in students.items()  
}  
# {'Alice': 86.33, 'Bob': 82.0, 'Charlie': 84.67}
```

```
# Filter: students scoring avg > 83  
top_students = {sid for sid, info in students.items() if info["marks"].values() > 83}
```




Data Structure Comparison — When to Use Which?

Feature	List []	Tuple ()	Set { }	Dict {k:v}
Ordered	✓ Yes	✓ Yes	✗ No	✓ Yes (3.7+)
Mutable	✓ Yes	✗ No	✓ Yes	✓ Yes
Duplicates	✓ Allowed	✓ Allowed	✗ No	Keys: No
Indexing	✓ Yes	✓ Yes	✗ No	By key
Hashing (as key)	✗ No	✓ Yes	✗ No	N/A
Lookup speed	O(n)	O(n)	O(1) ⚡	O(1) ⚡
Memory	Medium	Low ✓	Medium	High
Best use case	Sequence of items	Fixed records	Unique values	Key-value storage



Quick Rule: Need order+duplicates → List | Fixed data → Tuple | Unique values → Set | Named lookup → Dict



Searching, Sorting & the key Parameter

```
# — Membership Testing —————
fruits = ["apple","banana","mango","grape"]
print("mango" in fruits)      # True    - O(n) for list
print("orange" not in fruits) # True

scores = {90, 85, 72, 95, 60}
print(85 in scores)           # True    - O(1) for set ⚡

# — sorted() — returns NEW sorted list (original unchanged) —————
nums   = [5, 2, 8, 1, 9, 3]
asc    = sorted(nums)         # [1, 2, 3, 5, 8, 9]
desc   = sorted(nums, reverse=True) # [9, 8, 5, 3, 2, 1]
print(nums)                   # [5,2,8,1,9,3] ← original unchanged

# — key parameter — custom sort criteria —————
students = [("Alice",85),("Bob",72),("Eve",91),("Dan",78)]

by_score = sorted(students, key=lambda s: s[1])      # sort by score
by_name  = sorted(students, key=lambda s: s[0])      # sort alphabetically
top_first = sorted(students, key=lambda s: s[1], reverse=True)

# Real-world: sort dict list
employees = [
    {"name":"Alice","salary":75000},
    {"name":"Bob", "salary":55000},
    {"name":"Eve", "salary":90000},
```



Mini Project

Smart Inventory Management System — Using ALL 4 Data Structures



Smart Inventory Management System — Design

Add Product

Add new product with ID, name, category, qty, price, supplier

Update Inventory

Modify quantity or price of existing product by ID

Search Product

Find by product ID or name (case-insensitive)

Display Inventory

Show all products in formatted table view

Low Stock Alert

Warn when quantity falls below threshold (default: 10)

Out of Stock Alert

Highlight products with quantity = 0

Category Management

Track product categories using Sets

Inventory Report

Total items, total value, category-wise summary

Delete Product

Remove product from inventory by ID



Interview Questions & Answers

Q: List vs Tuple?

A: List is mutable, tuple is immutable. Tuples are faster and can be used as dict keys.

Q: Set vs Dictionary?

A: Set stores unique values. Dict stores key-value pairs. Both use hashing internally.

Q: Why are sets unordered?

A: Sets use a hash table internally. Hash values determine storage position, not insertion order.

Q: What is Hashing?

A: Converting data to a fixed-size integer via a hash function. Enables $O(1)$ lookup in sets and dicts.

Q: append() vs extend()?

A: `append(x)` adds `x` as ONE element. `extend(iterable)` adds ALL items from iterable individually.

Q: remove() vs pop()?

A: `remove(x)` finds by VALUE, raises `ValueError` if absent. `pop(i)` removes by INDEX, returns the element.

Q: Why tuples immutable?

A: Designed for fixed records. Immutability enables hashing, which allows tuples as dict keys/set members.

Q: Time complexity?

A: List search $O(n)$ | Dict/Set lookup $O(1)$ | List sort $O(n \log n)$ | List append $O(1)$ | List insert $O(n)$



Day 5 Summary — Key Takeaways

List

Mutable, ordered, allows duplicates. Use for sequences.

sorted()

Returns new sorted list. Use key=lambda for custom sort order.

Tuple

Immutable, ordered. Use for fixed data, dict keys, function returns.

Mutability

Mutable = can change. Immutable = cannot. Affects hashing & safety.

Set

Mutable, unordered, unique values. Use for deduplication & set math.

$O(1)$ Lookup

Sets and dicts use hashing → constant-time lookup regardless of size.

Dict

Mutable, key-value. Use for named lookup, records, counting.

Nested DS

Dicts of dicts, lists of tuples — build complex real-world structures.

Comprehension

[x for x in iterable if cond] — concise, Pythonic list/dict creation.

Mini Project

Smart Inventory uses all 4 DS: dict (records), set (categories), list+tuple.

[]

()

{ }

{:}

You've mastered Python Data Structures!

Practice every day — build projects, solve problems, ace interviews 🚀
